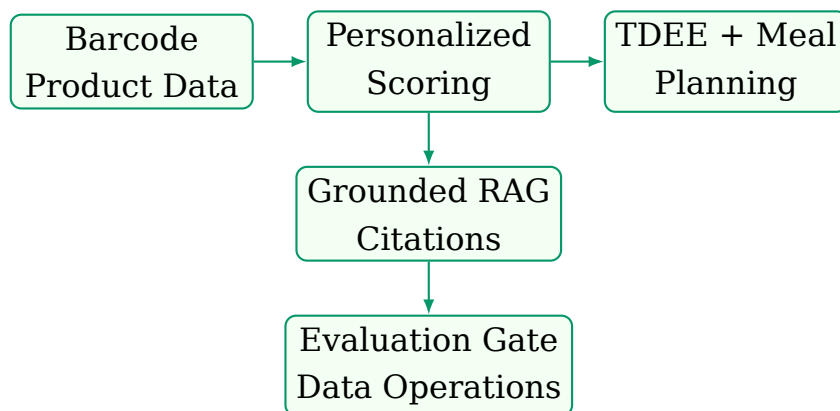


BÁO CÁO KỸ THUẬT

NUTRILENS AI

NỀN TẢNG AI VÀ DỮ LIỆU DINH DƯỠNG END-TO-END

Hybrid RAG, pgvector, Evaluation Gate và Data Operations



Dự án: NutriLens AI
Loại tài liệu: Báo cáo thiết kế, hiện thực và đánh giá hệ thống
Định hướng: AI Engineering / Data Engineering
Ngôn ngữ: Tiếng Việt
Ngày cập nhật: 11/07/2026

Nhóm phát triển NutriLens AI

Tóm tắt

Báo cáo trình bày thiết kế và hiện thực **NutriLens AI**, một nền tảng hỗ trợ người dùng hiểu nhãn thực phẩm, cá nhân hóa quyết định mua sắm, quản lý pantry, tính TDEE, lập kế hoạch bữa ăn và đặt câu hỏi dinh dưỡng có trích dẫn. Khác với một ứng dụng chỉ trình diễn giao diện, hệ thống được xây dựng như một dự án AI/Data Engineering end-to-end với hai data plane độc lập: Product Data và Knowledge Data.

Product Data lấy từ Open Food Facts, được chuẩn hóa, chấm điểm bằng luật xác định, cache có TTL và stale fallback. Knowledge Data đi qua meta-data validation, chunking theo heading, embedding, Knowledge Release bất biến, hybrid retrieval BM25/pgvector và Reciprocal Rank Fusion. Mọi release phải vượt Evaluation Gate trước khi publish; run thất bại được lưu như dữ liệu vận hành thay vì chỉ tồn tại trong log.

Phiên bản tại thời điểm báo cáo có 18 tài liệu tri thức, 21 evaluation cases ở các dataset, 14 SQLAlchemy models, 45 API operations và 42 automated tests. Frontend Next.js hỗ trợ desktop, mobile PWA và Control Plane quản trị riêng. Backend FastAPI chạy async với SQLite ở local/test và PostgreSQL 17 + pgvector ở production. Hệ thống được triển khai theo mô hình Netlify, Render và Neon.

Contents

Tóm tắt	i
1 Giới thiệu	1
1.1 Bối cảnh bài toán	1
1.2 Mục tiêu	1
1.3 Phạm vi	2
2 Khảo sát và phân rã miền	3
2.1 Cấu trúc monorepo	3
2.2 Bounded contexts	3
3 Kiến trúc hệ thống	5
3.1 Kiến trúc tổng thể	5
3.2 Request lifecycle	5
3.3 Chiến lược lỗi	6
4 Product Data và cá nhân hóa	7
4.1 Open Food Facts adapter	7
4.2 Deterministic scoring	7
4.3 TDEE và calorie target	7
4.4 Meal planning engine	8
5 Thiết kế Knowledge Data và RAG	9
5.1 Knowledge corpus	9
5.2 Ingestion và immutable release	9
5.3 Hybrid retrieval	10
5.4 Answer construction và abstention	10
6 Evaluation Gate và Data Operations	11
6.1 Benchmark	11
6.2 Các metric	11
6.3 Orchestration và lineage	12

6.4 Data quality, observability và analytics	12
7 Backend, frontend và mobile PWA	13
7.1 API surface	13
7.2 Identity và security	14
7.3 Frontend và mobile	14
8 Kiểm thử và kết quả	15
8.1 Chiến lược kiểm thử	15
8.2 Kết quả kiểm chứng	15
8.3 Các regression tiêu biểu	16
9 Triển khai và vận hành	17
9.1 Mô hình triển khai	17
9.2 Khởi động local	17
9.3 Data operations	18
10 Bảo mật, đạo đức và giới hạn	19
10.1 Threat model	19
10.2 Giới hạn	20
11 Hướng phát triển	21
12 Kết luận	22
A Ví dụ truy vấn đánh giá	23
B API mẫu	24

List of Figures

3.1 Kiến trúc logic của NutriLens AI	5
5.1 Pipeline tạo Knowledge Release	9

List of Tables

2.1	Các thành phần chính của NutriLens	3
6.1	Ngưỡng Evaluation Gate mặc định	11
7.1	Các nhóm API chính	13
8.1	Phạm vi kiểm thử tự động	15
9.1	Production deployment hiện tại	17
10.1	Các kiểm soát bảo mật chính	19
A.1	Các câu hỏi regression đại diện	23

Chapter 1

Giới thiệu

1.1 Bối cảnh bài toán

Nhãn dinh dưỡng chứa nhiều trường có ích nhưng khó diễn giải trong thời gian ngắn. Người dùng thường cần trả lời đồng thời ba nhóm câu hỏi: sản phẩm chứa gì, sản phẩm có phù hợp với hồ sơ cá nhân hay không, và một khái niệm dinh dưỡng có ý nghĩa gì. Một LLM thuần túy có khả năng diễn đạt tốt nhưng không phải nguồn sự thật cho barcode, calories hay dị ứng. Ngược lại, dữ liệu nhãn thô không đủ để giải thích bằng ngôn ngữ tự nhiên.

NutriLens AI giải quyết bài toán bằng cách phân chia trách nhiệm:

Open Food Facts = dữ liệu sản phẩm. Luật xác định = quyết định chấm điểm. RAG = kiến thức giải thích. Hồ sơ người dùng = ngữ cảnh cá nhân.

1.2 Mục tiêu

1. Xây dựng workflow scan-score-compare-save-plan hoàn chỉnh.
2. Giữ product facts tách biệt với dữ liệu sở hữu bởi từng người dùng.
3. Trả lời câu hỏi dinh dưỡng dựa trên corpus đã duyệt và có citation.
4. Từ chối trả lời khi retrieval không đủ bằng chứng.
5. Quản trị corpus bằng release, evaluation, publish và rollback.
6. Hỗ trợ PostgreSQL FTS + pgvector nhưng vẫn test được bằng SQLite.
7. Có pipeline lineage, data quality, observability và analytics marts.
8. Cung cấp trải nghiệm desktop, mobile PWA và admin riêng biệt.

9. Không biến ước tính dinh dưỡng hoặc TDEE thành chẩn đoán y khoa.

1.3 Phạm vi

Hệ thống tập trung vào dữ liệu nhãn thực phẩm đóng gói và kiến thức dinh dưỡng tổng quát. NutriLens không chẩn đoán bệnh, không kê chế độ điều trị và không thay thế chuyên gia. Độ chính xác product facts phụ thuộc mức đầy đủ của Open Food Facts. Meal plan và TDEE là ước tính hỗ trợ lập kế hoạch.

Chapter 2

Khảo sát và phân rã miền

2.1 Cấu trúc monorepo

Thành phần	Vai trò
apps/web	Next.js 16, TypeScript, barcode camera, dashboard, chat, meal planner, mobile PWA và Admin Control Plane.
apps/api	FastAPI, identity, product intelligence, profile/TDEE, personal data, RAG, orchestration và observability.
apps/api/data	Curated Markdown knowledge corpus có meta-data và provenance.
tests/evaluation	Dataset JSONL cho routing, grounding, fact coverage, abstention và security.
infra/migrations	Schema PostgreSQL, identity, RAG platform, pgvector và profile TDEE.
analytics	dbt-ready sources, semantic definitions và daily marts.
docs	Architecture, security, data platform, runbook, evaluation và deployment.

Table 2.1: Các thành phần chính của NutriLens

2.2 Bounded contexts

- **Identity:** đăng ký, đăng nhập, demo, JWT, cookie và user resolution.

- **Product Intelligence:** Open Food Facts, normalization, cache, scoring và comparison.
- **Personal Nutrition Data:** profile, TDEE, scans, pantry, favorites và meal plans.
- **Knowledge Corpus:** document, chunk, embedding, release và citation lineage.
- **RAG Evaluation:** benchmark, metrics, slices, confidence interval và gate.
- **Data Operations:** orchestration, quality, artifacts, rollback và analytics.

Nguyên tắc quan trọng là product cache được chia sẻ để giảm gọi upstream, nhưng mọi quyết định cá nhân đều lọc theo `user_id`. Truy vấn tài nguyên dùng đồng thời resource ID và owner ID để không làm lộ sự tồn tại dữ liệu của người khác.

Chapter 3

Kiến trúc hệ thống

3.1 Kiến trúc tổng thể

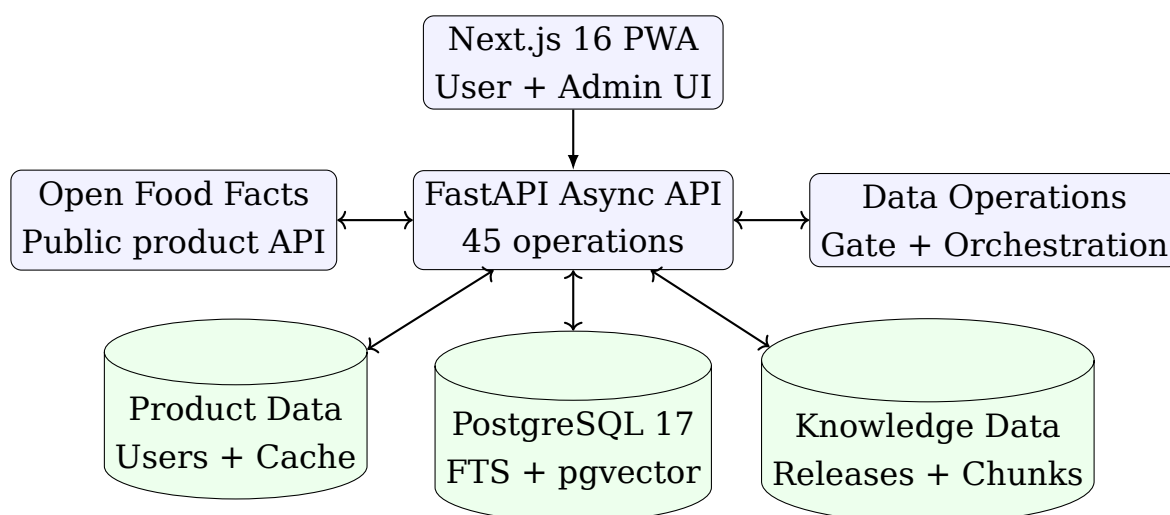


Figure 3.1: Kiến trúc logic của NutriLens AI

3.2 Request lifecycle

1. Middleware tạo hoặc tiếp nhận request ID và bắt đầu đo latency.
2. Rate limiter phân loại endpoint nhạy cảm.
3. CORS và JWT cookie xác định caller.
4. Router gọi domain service và truy vấn owner-scoped.
5. Response được gắn security headers, request ID và timing.
6. Chat ghi audit hash, route, abstention, citation count và latency.

3.3 Chiến lược lỗi

Input không hợp lệ trả 422; thiếu xác thực trả 401; tài nguyên không thuộc user trả 404; upstream lỗi trả stale cache hoặc 502; evidence yếu khiến RAG abstain. Readiness được tách khỏi startup nặng để container có thể mở port sớm trên nền tảng free tier.

Chapter 4

Product Data và cá nhân hóa

4.1 Open Food Facts adapter

Barcode được validate trước khi gọi upstream. Response được chuẩn hóa về tên, brand, categories, ingredients, allergens, additives, nutriments, Nutri-Score, Eco-Score và image. Cache dùng TTL; khi upstream tạm lỗi, record cũ vẫn được trả để tránh biến sự cố mạng thành outage toàn bộ workflow.

4.2 Deterministic scoring

Scoring engine phân tích calories, sugar, sodium, saturated fat, protein, fiber, allergens, diet, additives và Nutri-Score. Luật xác định được ưu tiên trước LLM vì ba lý do: reproducible, dễ test và có thể giải thích field nào đã làm thay đổi score. Response luôn mang good points, warnings, missing fields và disclaimer.

4.3 TDEE và calorie target

Hệ thống dùng phương trình Mifflin-St Jeor [2] để ước tính BMR:

$$\text{BMR} = 10W + 6.25H - 5A + S, \quad (4.1)$$

trong đó W là cân nặng kg, H là chiều cao cm, A là tuổi; $S = 5$ cho nam và $S = -161$ cho nữ. TDEE được ước tính bởi:

$$\text{TDEE} = \text{BMR} \times f_{\text{activity}}. \quad (4.2)$$

Mục tiêu giảm L kg/tuần tạo deficit yêu cầu $L \times 7700/7$ kcal/ngày. Service giới hạn deficit ở 25% TDEE và calorie floor tham khảo, đồng thời cảnh báo

khi mục tiêu vượt 1% cân nặng mỗi tuần. Kết quả được đưa trực tiếp sang Meal Planner nhưng vẫn được mô tả là ước tính dân số, cần hiệu chỉnh theo theo dõi thực tế.

4.4 Meal planning engine

Mỗi món có meal type, goals, diets, ingredients, aliases, allergens, kcal, protein, fiber và estimated cost. Candidate được loại theo diet/dị ứng trước, sau đó chấm điểm theo mục tiêu, pantry match, lịch sử lặp và budget. Shopping list được aggregate từ nguyên liệu thật và bỏ các item khớp pantry. Thiết kế này thay thế bản prototype từng lặp cùng một món cho mọi bữa.

Chapter 5

Thiết kế Knowledge Data và RAG

5.1 Knowledge corpus

Corpus tại thời điểm báo cáo gồm 18 tài liệu Markdown về độc nhãn, sugar, sodium, protein, fiber, saturated fat, allergens, additives, food safety, vegetarian/vegan, diabetes, hypertension, children nutrition và các câu hỏi đời thường như trứng, cá, yogurt-yến mạch, cà phê. Metadata bắt buộc gồm authority, source URL, jurisdiction, risk level, effective/review dates, evidence level, domains và status.

5.2 Ingestion và immutable release

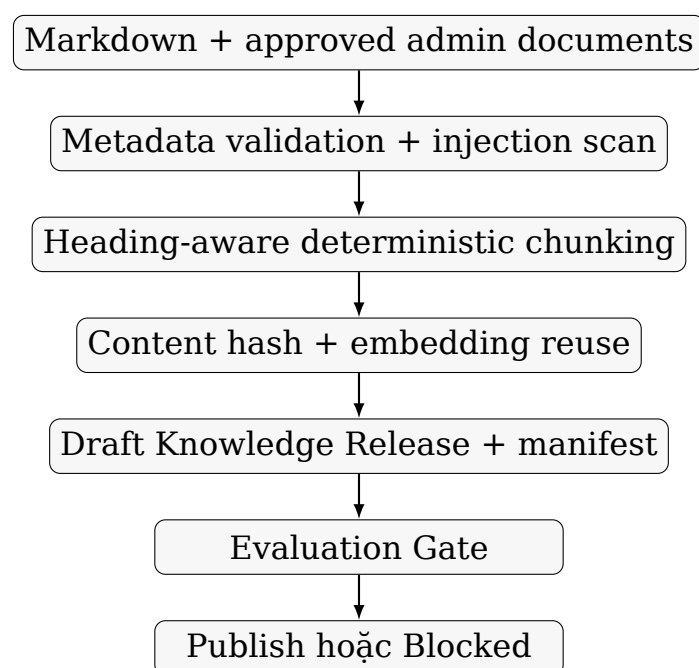


Figure 5.1: Pipeline tạo Knowledge Release

Chunk giữ source filename/title/URL, document ID, heading path, content hash, token count, metadata, embedding và embedding model. Embedding cũ được reuse khi cặp content_hash + embedding_model không đổi. Manifest hash làm release reproducible và hỗ trợ artifact export.

5.3 Hybrid retrieval

Local mode dùng BM25 và feature-hash embedding; production dùng PostgreSQL FTS và pgvector để lấy candidate. BM25 cho token t trong document d :

$$\text{BM25}(t, d) = \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b \frac{|d|}{\text{avgdl}})}. \quad (5.1)$$

Lexical rank và semantic rank được hợp nhất bằng Reciprocal Rank Fusion:

$$\text{RRF}(d) = \frac{w_l}{k + r_l(d)} + \frac{w_s}{k + r_s(d)}. \quad (5.2)$$

Title và heading được boost. Semantic similarity chỉ refine candidate có lexical evidence để tránh collision từ feature hashing đưa chunk không liên quan vào câu trả lời. Alias tiếng Việt hỗ trợ từ có dấu, không dấu và các token gần quan trọng như “cá” hoặc “xơ”.

5.4 Answer construction và abstention

Runtime phân loại route thành RAG, product hoặc mixed. Product-specific facts không được suy từ corpus. Retrieval hit phải vượt relevance floor; nếu không, API trả abstained response. Citation chứa source, title, URL, snippet, chunk ID, heading path, lexical score, semantic score và fused score.

Chapter 6

Evaluation Gate và Data Operations

6.1 Benchmark

Repository có 21 JSONL cases, trong đó core benchmark hiện nạp 19 cases từ các nhóm nutrition, mixed product, abstention, benchmark v2 và everyday food. Hai case còn lại phục vụ product/security dataset riêng. Mỗi case có thể khai báo expected route, expected source, required facts và should abstain.

6.2 Các metric

Metric	Ý nghĩa	Gate
Route accuracy	Tỷ lệ chọn đúng RAG/product/mixed.	= 1.00
Abstain accuracy	Tỷ lệ từ chối đúng khi thiếu evidence.	= 1.00
Source recall	Expected source xuất hiện trong citations.	≥ 0.80
Mean reciprocal rank	Expected source xuất hiện càng sớm càng tốt.	≥ 0.70
Required fact coverage	Tỷ lệ required facts có trong answer.	≥ 0.90
Retrieval latency p95	Phân vị 95 của retrieval latency.	≤ 250 ms

Table 6.1: Ngưỡng Evaluation Gate mặc định

Evaluator còn lưu slice metrics, Wilson confidence intervals, retrieval trace và trend so với run trước. Nếu bất kỳ metric nào vi phạm threshold, release giữ trạng thái draft và orchestration run chuyển sang blocked.

6.3 Orchestration và lineage

Pipeline chính gồm ingest, evaluation gate và publish. Mỗi stage lưu status, duration, input/output count, release ID, evaluation run ID và failures. Core logic không phụ thuộc framework; Prefect là adapter tùy chọn. Cách này cho phép chạy cùng workflow từ CLI, API admin hoặc orchestrator.

6.4 Data quality, observability và analytics

Quality report theo dõi product completeness, stale ratio, missing fields, release count, duplicate hashes, empty embeddings và pipeline reliability. Endpoint `/metrics` cung cấp Prometheus exposition. Admin snapshot tổng hợp citation coverage, abstention, latency và active release. Thư mục `analytics/` chứa dbt-ready marts cho RAG và pipeline theo ngày; semantic catalog mô tả metric, grain, owner và freshness.

Chapter 7

Backend, frontend và mobile PWA

7.1 API surface

Nhóm endpoint	Chức năng
/api/auth/*	Register/signup, login, demo, current session, logout.
/api/products/*	Scan, lookup, deterministic score và comparison.
/api/profile	Profile preferences, body/activity inputs và TDEE.
/api/pantry, /api/favorites	User-owned shopping and storage workflows.
/api/meal-plan/*	Generate và retrieve personalized meal plans.
/api/chat/stream	Grounded answer, route, citations và retrieval trace.
/api/rag/search	Retrieval debugging có score breakdown.
/api/admin/*	Documents, releases, gate, rollback, pipeline, quality và marts.
/health/*, /metrics	Container health và observability.

Table 7.1: Các nhóm API chính

7.2 Identity và security

Password được hash bằng Argon2; access token dùng JWT trong HttpOnly cookie. Production yêu cầu secret mạnh, secure cookie và CORS origin giới hạn. Admin operation yêu cầu X-Admin-Key. Raw prompt không được lưu; audit chỉ giữ hash một chiều. Prompt-injection patterns được kiểm tra khi ingest admin document. Response dùng CSP, frame denial, content-type protection và request ID.

7.3 Frontend và mobile

Next.js App Router cung cấp dashboard, scanner, product detail, compare, chat, pantry, favorites, meal planner, profile và admin. Mobile UI có sticky header, bottom navigation, bottom sheet, safe-area cho iOS, touch target, camera viewport ổn định và kiểm tra horizontal overflow ở viewport 390x844. Manifest hỗ trợ standalone PWA và shortcuts cho scan, chat, meal planner.

Chapter 8

Kiểm thử và kết quả

8.1 Chiến lược kiểm thử

Nhóm	Nội dung
Auth integration	Cookie lifecycle, demo, admin access và cross-user isolation.
Product data	Cache freshness, timezone, stale fallback và upstream failure.
Scoring	Sugar, diet, allergen và missing-data behavior.
Meal/TDEE	Diversity, vegan/exclusion, budget, calorie guardrail và công thức BMR.
RAG core	Tokenization, embedding, BM25/RRF, citations và abstention.
Data platform	Release lineage, reuse, gate, rollback, orchestration, quality và artifacts.
Frontend	ESLint, TypeScript, Next.js production build.
Browser E2E	Authenticated portfolio flow và mobile navigation/overflow.

Table 8.1: Phạm vi kiểm thử tự động

8.2 Kết quả kiểm chứng

Tại thời điểm lập báo cáo:

- 42/42 Python tests vượt qua;
- Ruff API lint vượt qua;

- ESLint không có error (còn một warning font từ Next.js);
- Next.js 16 production build thành công với 12 routes;
- 2/2 mobile E2E tests vượt qua ở viewport 390x844;
- Evaluation Gate thử nghiệm với corpus mở rộng đã publish thành công;
- một run kiểm chứng đạt source recall 0.8824, MRR 0.7353 và required fact coverage 0.9688, đều vượt threshold tương ứng.

Các metric cuối là kết quả của một evaluation run cụ thể, không phải hằng số được hard-code. Mỗi thay đổi corpus/retrieval phải tạo run mới và đi qua gate.

8.3 Các regression tiêu biểu

1. Meal planner ban đầu dùng một pool nhỏ nên lặp yogurt cho nhiều bữa; được thay bằng catalog có slot, macro, ingredient và diversity penalty.
2. Vegan đồng thời tránh soy làm hết candidate bữa sáng; test phát hiện và catalog được bổ sung lựa chọn oats/chia không soy.
3. Feature-hash semantic collision đưa đoạn trứng/cà phê vào câu protein; semantic rank được giới hạn trong lexical-grounded candidates.
4. Tokenizer bỏ từ ngắn dưới ba ký tự, làm mất “cá” và “xơ”; alias-aware token retention giúp Evaluation Gate tăng source recall.
5. PostgreSQL không chấp nhận phép toán giữa datetime aware/naive trong product cache; timestamp contract được chuẩn hóa cho asyncpg.
6. Container từng bị health check trước khi mở port; schema initialization được đưa ra background và health endpoint hỗ trợ preflight.

Chapter 9

Triển khai và vận hành

9.1 Mô hình triển khai

Thành phần	Nền tảng	Vai trò
Frontend	Netlify	Next.js PWA, redirects cùng origin tới API.
Backend	Render	Root Dockerfile, Uvicorn, health/readiness.
Database	Neon	Managed PostgreSQL, SSL pooler và pgvector.

Table 9.1: Production deployment hiện tại

9.2 Khởi động local

```
1 python -m venv .venv
2 source .venv/bin/activate
3 pip install -r apps/api/requirements.txt
4 npm ci
5
6 npm run dev:api
7 # terminal thu hai
8 npm run dev:web
```

Full stack PostgreSQL:

```
1 docker compose up --build
2 curl http://localhost:8000/health/ready
3 curl http://localhost:8000/metrics
```

9.3 Data operations

```
1 npm run data:pipeline
2 npm run data:evaluate
3 npm run data:quality
4 npm run data:analytics
5 npm run data:search -- "An trung co tot khong?"
6 npm run validate
7 npm run test:e2e
```

Sau deploy corpus mới, operator phải chạy release pipeline ở Admin Control Plane. Runtime tiếp tục dùng published release cũ cho đến khi candidate mới vượt gate; đây là hành vi có chủ đích.

Chapter 10

Bảo mật, đạo đức và giới hạn

10.1 Threat model

Các rủi ro gồm prompt injection, poisoned document, cross-user data access, secret exposure, insecure cookie, upstream data sai/thiếu, overconfident health answer và abuse admin endpoint.

Kiểm soát	Hiện thực
Identity	Argon2, JWT, HttpOnly/Secure cookie, active-user check.
Ownership	Mọi personal resource query dùng user_id.
Admin boundary	Control Plane riêng và X-Admin-Key.
Prompt privacy	SHA-256 question hash; không lưu raw question trong audit.
Corpus safety	Metadata validation, injection scan, approved status.
Grounding	Citation lineage, lexical evidence gate và abstention.
Operational security	CORS allowlist, rate limit, security headers, request ID.
Release safety	Evaluation Gate không bypass, immutable release và rollback.

Table 10.1: Các kiểm soát bảo mật chính

10.2 Giới hạn

Open Food Facts là dữ liệu cộng đồng nên có thể thiếu hoặc sai. Feature hashing phù hợp reproducible test nhưng không thay thế neural embedding ở quy mô lớn. Corpus 18 tài liệu và core benchmark 19 cases vẫn nhỏ so với production health assistant. TDEE, deficit và meal-plan macro là ước tính. Hệ thống không phù hợp để chẩn đoán, điều trị hoặc xử lý emergency.

Chapter 11

Hướng phát triển

1. Mở rộng benchmark lên hàng trăm câu, thêm adversarial và multilingual slices.
2. Thử nghiệm sentence-transformer tiếng Việt và reranker cross-encoder.
3. Thêm offline/online retrieval evaluation và A/B experiment registry.
4. Dùng Alembic cho migration lifecycle thay vì chỉ bootstrap idempotent.
5. Triển khai dbt warehouse, freshness test và data contract CI.
6. Thêm object storage cho artifacts và retention policy.
7. Thêm alert dashboard cho gate failure, latency và citation coverage.
8. Mở rộng meal catalog, serving scaling và optimizer theo macro constraints.
9. Hiệu chỉnh TDEE bằng chuỗi cân nặng quan sát được với privacy controls.
10. Bổ sung nguồn hướng dẫn dinh dưỡng chính thống theo jurisdiction Việt Nam.

Chapter 12

Kết luận

NutriLens AI chứng minh một ứng dụng AI đáng tin cậy cần nhiều hơn một chat box. Giá trị cốt lõi nằm ở data contracts, deterministic domain logic, retrieval có lineage, evaluation gate, pipeline observability và failure behavior rõ ràng. Việc tách Product Data khỏi Knowledge Data cho phép hệ thống vừa phản hồi dữ liệu barcode cụ thể, vừa giải thích kiến thức có nguồn mà không trộn hai loại sự thật.

Kiến trúc hiện tại đủ để trình diễn năng lực end-to-end của AI/Data Engineer: ingestion, normalization, storage, retrieval, evaluation, orchestration, analytics, API, frontend và deployment. Đồng thời, các disclaimer và abstention giữ phạm vi của sản phẩm ở mức hỗ trợ thông tin thay vì tạo cảm giác chẩn đoán y khoa.

Appendix A

Ví dụ truy vấn đánh giá

Câu hỏi	Nguồn/route mong đợi
Ăn trứng có tốt không?	eggs_and_health.md, RAG.
Uống cà phê mỗi ngày có sao không?	coffee_and_caffeine.md, RAG.
Yogurt và yến mạch có phù hợp cho bữa sáng?	Breakfast guidance, RAG.
Tôi dị ứng sữa, sản phẩm này dùng được không?	Food allergens, mixed.
Protein trên nhãn dinh dưỡng có ý nghĩa gì?	Protein basics, RAG.
Một chất bí mật XZ-991 chữa bệnh gì?	Abstain.

Table A.1: Các câu hỏi regression đại diện

Appendix B

API mẫu

```
1 curl -X POST http://localhost:8000/api/products/scan \  
2   -H 'Content-Type: application/json' \  
3   -d '{"barcode":"737628064502"}'  
4  
5 curl -X POST http://localhost:8000/api/chat/stream \  
6   -H 'Content-Type: application/json' \  
7   -d '{"question":"An trung co tot khong?","barcode":null}'  
8  
9 curl -X POST http://localhost:8000/api/profile/tdee \  
10  -H 'Content-Type: application/json' \  
11  -d '{"biological_sex":"male","age":30,"height_cm":175,  
12      "weight_kg":75,"activity_level":"moderate",  
13      "target_weight_loss_kg_week":0.5,"goal":"weight_loss"}'
```

Bibliography

- [1] Open Food Facts, *Open Food Facts API Documentation*. <https://openfoodfacts.github.io/openfoodfacts-server/api/>.
- [2] M. D. Mifflin et al., *A New Predictive Equation for Resting Energy Expenditure in Healthy Individuals*, American Journal of Clinical Nutrition, 51(2), 1990. <https://pubmed.ncbi.nlm.nih.gov/2305711/>.
- [3] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020. <https://arxiv.org/abs/2005.11401>.
- [4] G. V. Cormack, C. L. A. Clarke, S. Buettcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR, 2009.
- [5] pgvector, *Open-source Vector Similarity Search for Postgres*. <https://github.com/pgvector/pgvector>.
- [6] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*. <https://genai.owasp.org/llm-top-10/>.
- [7] National Institute of Diabetes and Digestive and Kidney Diseases, *Body Weight Planner*. <https://www.niddk.nih.gov/bwp>.